

课题10-长程Agent

Contents

1 课题10 开题简报 · 长程 Agent 与 self-judge 1

1.1 一、研究背景 1

1.2 二、研究问题 1

1.3 三、攻关线索 2

1.4 四、里程碑 2

1.5 五、交付物 2

1.6 六、讲课锚点 3

1.7 七、代码级提问示例 3

1.8 八、风险与提醒 3

1 课题10 开题简报 · 长程 Agent 与 self-judge

模块：M8 | 周次：W14 (12-14→12-20) | 算力：T2
Kaggle 2×T4, 计划 8 GPU·小时 唐杰作业对应：⑤ 训长程
Agent, 鼓励 self-judge loop | 判分来源：自建（消融实验 +
轨迹日志） 手写：R10 (self-judge 协议设计)

1.1 一、研究背景

从”回答问题”到”完成任务”的跨越，靠的是 **harness**（外壳）：工具调用、上下文管理、失败重试、自我评估。Karpathy 与李宏毅都专讲 **context engineering**；nanochat 里 `execution.py` 就是一个最小的”能执行代码的 Agent 外壳”。本课题的落点是长程任务：多步串联后误差累积，以及 **self-judge**（自我评判）到底有没有用。

1.2 二、研究问题

多步任务中，失败主要来自模型能力还是来自 **harness** 设计？加上 **self-judge loop** 后成功率提升多少——还是反而下降？

1.3 三、攻关线索

1. 最小 **ReAct**: 不用任何框架, 手写 Thought $\rightarrow$ Action $\rightarrow$ Observation 循环 (三四十行就能跑)
2. 工具设计: 先给两个工具 (计算器 / 受限代码执行)。工具接口的清晰度往往比模型智力更决定成败
3. 长程任务构造: 把课题09 的题改造成“需要 3-5 步”的复合任务 (如: 先算 $\rightarrow$ 再验 $\rightarrow$ 再格式化)
4. 失败分类: 格式错 / 工具调错 / 目标漂移 (忘了原始要求) / 循环打转 / 提前放弃
5. **self-judge** 的两种形态:
 - 自评重试: 模型给自己打分, 低于阈值就重做 (rasbt ch5 的 self-refinement)
 - 自评过滤: 生成多个候选, 自己挑最好的 (best-of-N 的变体)
6. 关键实验 (消融): 同样的任务集, 三组对照 = 无 self-judge / 自评重试 / 自评过滤
7. 警惕: self-judge 会放大原有错误 (模型判不出自己错在哪, 反而确信) $\rightarrow$ 必须留失败案例

1.4 四、里程碑

- ☐ M1 手写最小 ReAct 循环 (无框架), 跑通单步任务
- ☐ M2 加入第二个工具 + 多步任务集 (≥ 30 个任务)
- ☐ M3 轨迹记录规范: 每步的 thought/action/observation 落盘 (可复现)
- ☐ M4 基线成功率 + 失败五分类 (每类给 2 个真实案例)
- ☐ M5 **self-judge** 消融: 三组对照, 给出成功率差与误差棒
- ☐ M6 反例收集: 找出 ≥ 3 个“self-judge 导致更差”的案例并解释机制
- ☐ M7 一页报告 + 知识卡

1.5 五、交付物

交付物	位置	验收
ReAct 循环	手写/	可跑通多步任务
self-judge 协议	手写/ (R10)	有明确的判定时机与阈值设计说明
轨迹日志	证据/trajectories/	每任务一份, 含步数与结果
消融数据	证据/	三组成功率 + 误差棒
报告	报告.md	必须回答“何时 self-judge 有害”

1.6 六、讲课锚点

- 讲义：M8-智能体.md (W13 展开)
- 对标：李宏毅《Context Engineering》专讲 (Agent 时代的关键技术) · Berkeley LLM Agents MOOC (ReAct 原始论文那讲) · HF Agents Course (三框架对照 + 观测性/评测 bonus unit) · rasbt ch5 (self-refinement) · nanochat/execution.py+tests/test_execution.py (沙箱执行的最小实现, 可读接口不可抄实现)
- 搭桥：M5 的 KV cache (长上下文的显存代价) → 为什么长程 Agent 必须做上下文压缩

1.7 七、代码级提问示例

1. 长程任务失败率随步数指数上升。分解：是每步错误率问题，还是错误会累积传播？
2. self-judge 什么时候一定无效？（提示：当模型无法区分“看起来对”和“真的对”时）
3. 上下文压缩（截断/摘要）会引入什么新失败模式？
4. 为什么“工具返回值要结构化”能显著提升成功率？（从 token 分布角度解释）
5. 【下注】预测 self-judge 在简单任务 vs 困难任务上的效果差异，先写再测。

1.8 八、风险与提醒

- 🚫 不要用框架起步：手写一遍 ReAct 再去看 LangGraph/smolagents, 否则学不到 harness 的本质。
- ⚠ self-judge 会显著增加 token 消耗与时间 → 算力预算要写进实验卡。
- ⚠ 轨迹日志可能很大 → 只保留结构化摘要 (thought 截断 + 结果), 避免污染仓库体积。